School of Future Tech

Case Study Report

(HTML,CSS,JavaScript)

on

MEME BATTLE – Caption Challenge Game

by

Aditya Arvind Kadam

150096725122

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

MemeBattle is a single-player, browser-based game where users write funny captions for classic internet memes. After each caption is submitted, an AI-simulated voting system assigns likes, dislikes, and LOLs. The final score is calculated from those votes combined with a time bonus and a caption length bonus. Players compete across 5 rounds, and their total score is saved to a persistent leaderboard stored in the browser's localStorage.

The project started as a single-file proof of concept and was then refactored into a clean 5-file structure with separated concerns — 2 HTML pages, 1 shared CSS file, and 2 JavaScript files. The goal was to build something that looks and feels like a real product — animated timers, score popups, a dark arcade aesthetic — without touching a server or a database.

2 WEBPAGES	5 FILES	9 MEMES	5 ROUNDS

Goals & Purpose

- Build a fun, self-contained browser game with zero backend
- Demonstrate clean HTML/CSS/JS architecture split across multiple files
- Persist leaderboard data across sessions using localStorage
- Deliver a polished, arcade-aesthetic UI using only vanilla technologies

Key Features

Feature	Description
9 Meme Templates	Classic memes pulled from imgflip.com, randomised with no repeats per session
30-Second Timer	SVG ring countdown, turns red and pulses at ≤10 seconds
AI Vote Simulation	Likes, dislikes and LOLs generated from caption quality and length
Score Formula	(Likes×10 + LOLs×8 − Dislikes×5) + TimeLeft×3 + LengthBonus(+50 if >30 chars)
Rank Tiers	Rookie → Rising Star → Meme Pro → Meme Legend (based on total score)
Achievements	Caption God, Consistent, On Fire, Hot Start — awarded at game end
Leaderboard	Top 10 displayed, top 20 stored in localStorage, persists across sessions
Responsive Design	Mobile breakpoints at 860px and 560px, hero collapses to single column

2. Problem Statement / Case Background (Abstract)

Background

Most browser games of this type rely on a login system, a backend for storing scores, or a heavy JavaScript framework. The challenge here was to build something genuinely functional and polished using nothing but the browser's native APIs — localStorage for persistence, the DOM for all rendering, and a scoring algorithm for simulated player engagement. The additional constraint was that all 5 files must work when opened directly from a folder, with no build step or local server.

Abstract

MemeBattle is a five-round, browser-only caption game built with vanilla HTML, CSS, and JavaScript — no frameworks, no server, no build step. Each round loads a random meme from imgflip.com and gives the player 30 seconds to write a caption. When submitted, a simulated crowd votes on it, generating likes, dislikes, and LOLs based on caption length. Those votes feed into a scoring formula that also rewards speed and longer captions. Scores persist across sessions via localStorage and appear on a leaderboard shared between the two pages — the only communication channel the project uses.

---

3. Problem Statement / Case Study Design

The system is designed as a static multi-page site. Both pages share one CSS file and communicate exclusively through localStorage — no server, no API, no shared JS state. Navigation between pages uses window.location.href.

File Structure

File	Lines	Purpose
index.html	80	Home page — hero, how-to timeline, leaderboard, forum
play.html	98	Play page — username screen, game, voting panel, game-over screen
style.css	600	All shared styles — linked by both HTML pages
home.js	54	Home page logic — leaderboard render, stats update, clear function
play.js	121	Full game engine — timer, scoring, vote simulation, localStorage save

How Files Link Together

Both HTML files share the same CSS and load their own JS. The two pages communicate exclusively through localStorage — no server, no API, no shared JS state.

From		To
index.html	→	style.css (via <link rel="stylesheet">)
index.html	→	home.js (via <script src>)
play.html	→	style.css (via <link rel="stylesheet">)
play.html	→	play.js (via <script src>)
play.js	→	localStorage (writes scores, game count)
home.js	→	localStorage (reads leaderboard, stats)

Game Flow

Username entry → Random meme loads → 30s timer starts → Caption submitted → Votes simulated → Round score shown → Repeat ×5 → Results saved to leaderboard

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Score Formula

Each round produces a score calculated as:

```
pts = (likes × 10) + (LOLs × 8) − (dislikes × 5) + (timeLeft × 3) + lengthBonus

lengthBonus = 50  if caption.length > 30,  else 0
Minimum score per round = 0  (negative totals are floored)
```

Vote Simulation Algorithm

Votes are procedurally generated in submitCaption() based on caption length. The chain is: caption length → quality score → likes → LOLs/dislikes.

```
// Step 1 — derive quality from caption length
quality = caption.length > 20 ? rand(0.4–1.0) : rand(0–0.4)

// Step 2 — generate random voter base
base = random integer 20–100

// Step 3 — calculate votes
likes    = floor(base × quality)
dislikes = floor(base × (1 − quality) × 0.6)

// Step 4 — LOLs derived from likes, NOT directly from length
LOLs = floor(likes × rand(0.3–1.0))
```

Rank Tiers

Rank	Score Threshold	Badge Colour
🔱 **MEME LEGEND**	2000+	Gold
💜 **MEME PRO**	1500–1999	Purple
🌱 **RISING STAR**	1000–1499	Green
🐣 **ROOKIE**	0–999	Grey

Achievements

Achievement	Condition	Notes
🔥 **Caption God**	Best single round ≥ 800	Max of roundScores ≥ 800
⚡ **Consistent**	All rounds ≥ 300	Min of roundScores ≥ 300
🎯 **On Fire**	3+ rounds ≥ 600	Filter roundScores ≥ 600, length ≥ 3
💥 **Hot Start**	Round 1 score ≥ 700	roundScores[0] ≥ 700
🎮 **Participated**	Fallback — always awarded	Ensures player always gets at least one

Technology Stack

Area	Detail
Language	Vanilla JavaScript (ES6+), HTML5, CSS3 — no frameworks or bundlers
Fonts	Orbitron (display) + DM Sans (body) — loaded via Google Fonts CDN
Storage	localStorage — JSON leaderboard array + total games integer counter
Animation	CSS @keyframes + SVG stroke-dashoffset for the countdown ring
Images	Meme templates served directly from imgflip.com via <img src>
Environment	Any modern browser (Chrome, Firefox, Safari, Edge) — no install, no server

5. Problem Statement / Case Study Implementation Details and Snapshots

JavaScript — home.js

54 lines. Handles everything on the home page. Runs automatically when the page loads.

Function	What it does
refreshHome()	Reads localStorage and fills the leaderboard table and header stat pills. Called on page load automatically.
clearLB()	Asks for confirmation, then removes memeLB from localStorage and re-renders the empty leaderboard.
getRank(score)	Returns the rank object matching the given score (Rookie / Rising Star / Meme Pro / Meme Legend).
getLB()	Safely parses the leaderboard array from localStorage. Returns empty array on error.
toast(msg)	Shows a floating notification at the bottom of screen for 2.5 seconds.

JavaScript — play.js

121 lines. The full game engine. Handles game state, all user interactions, scoring, vote simulation, and saving results.

Game State Object

All mutable game data is stored in a single object G, reset at the start of every game:

```
const G = {
  username:    '',    // Player's callsign
  round:       1,     // Current round (1-5)
  score:       0,     // Cumulative total score
  usedMemes:   [],    // IDs of memes already shown this session
  roundScores: [],    // Score per round (used for achievements)
  timer:       null,  // setInterval reference — cleared on submit
  timeLeft:    30,    // Seconds remaining on current round
  submitted:   false  // Prevents double-submit on the same round
}
```

Functions in play.js

Function	What it does
startGame()	Validates username, resets G state, hides username-screen, shows game-screen, calls loadRound()
loadRound()	Picks a random meme (no repeats), updates all game-screen elements, calls startTimer()

startTimer()	Clears existing interval, starts 1s countdown. Updates SVG ring strokeDashoffset and applies danger class at ≤10s
submitCaption()	Stops timer, simulates votes based on caption quality/length, calculates points, shows votingPanel after 900ms delay
nextRound()	If round < 5: increments round, calls loadRound(). If round = 5: calls endGame()
endGame()	Shows game-over screen, calculates rank and achievements, saves to localStorage, shows leaderboard position
resetGame()	Hides game/game-over screens, shows username-screen for a new game
countUp(id, target)	Animates a number from 0 to target over 800ms using requestAnimationFrame
spawnPopup(text)	Creates a floating score popup element at 45% top, appends to body, removes after 1.7s

Data & localStorage

The game uses the browser's built-in localStorage as its database. No server, no backend, no API calls. Data persists across sessions until the user clears browser storage or presses RESET.

Key	Type	Contents
memeLB	JSON string	Array of up to 20 leaderboard entry objects, sorted by score descending
totalGames	String (number)	Integer count of total games ever played, incremented at endGame()

6. Problem Statement / Case Study Results and Conclusion

Findings

- The vote simulation rewards longer, more creative captions — quality and randomness are balanced so the scoring feels fair without the algorithm being obvious to the player.
- Splitting from a 416-line minified single file into 5 files with clear responsibilities made the codebase significantly easier to maintain and extend.
- CSS custom properties made visual iteration fast — a palette change in :root updated the entire UI in one edit.
- localStorage is sufficient for a solo leaderboard but is browser-local. Scores don't survive clearing browser data or switching devices.
- Loading meme images from imgflip.com introduces an external dependency — the game breaks if the CDN is unreachable.

Conclusion

MemeBattle demonstrates that a polished, interactive browser game can be built without a framework, a backend, or a build pipeline. The project covers a realistic frontend development workflow: prototyping in a single file, refactoring into a structured multi-file project, applying a consistent design system via CSS variables, and documenting the full codebase. The architecture is deliberately simple — every part is easy to trace, extend, or replace. Real multiplayer, a backend leaderboard, or a mobile-native version would all be natural next steps.

7. References

- MDN Web Docs — localStorage API, SVG stroke-dashoffset animation, CSS custom properties
- Google Fonts — Orbitron and DM Sans typefaces (fonts.google.com)
- imgflip.com — meme template image hosting
- CSS-Tricks — CSS Grid layout, flexbox patterns, and keyframe animation references
- docx npm package — JavaScript library used for generating this report document